

CONTENTS

Table of Contents

01	What is CreBot? (In Plain Words)	3
02	Who Is It For? & The Problem It Solves	4
03	Key Features	5
04	How CreBot Works — The Big Picture	6
05	Technology Stack Explained Simply	7
06	Authentication Flow (Clerk + JWT)	8
07	Frontend Architecture — Every Page & Component	9
08	Backend Architecture — Every Route & Service	11
09	Database Schema (Supabase + pgvector)	13
10	The Embeddable Widget	14
11	Complete Folder & File Structure	15
12	Step-by-Step: From Upload to Answer	16
13	Security & Privacy	17
14	Access Control & Team Roles	18
15	Rate Limits, BYOK & Pricing	19
16	Roadmap — What's Coming Next	20
17	Quick Start Guide	21
18	Glossary — Every Technical Word Explained	22

How to read this document: Every section is written in plain, simple English first. Whenever a technical word shows up (like "API" or "vector"), you don't need to stop — just keep reading, and flip to the **Glossary (Section 18)** whenever you want the plain-English definition of any term used in this document.

What is CreBot?

The elevator pitch, explained the way you'd explain it to a friend who has never written a line of code.

CreBot is a website where any business owner can build their own AI chatbot in a few minutes — without writing any code. You upload your own documents (like your FAQ, product manual, or company policies), and CreBot turns that into a smart chat assistant that can answer questions about *only* that information.

Then you copy a small piece of code and paste it into your own website. A chat bubble appears in the corner, and any visitor can now click it and start asking questions — and get instant, accurate answers based on your documents, 24 hours a day.

The One Rule CreBot Never Breaks: "Zero-Speculation"

Most general AI chatbots (like a generic assistant) will try to guess an answer even if they don't actually know it. This is called **hallucination** — the AI makes something up that sounds correct but isn't. CreBot is built specifically to avoid this. Its core philosophy is called **Zero-Speculation RAG**:

In simple words: CreBot's chatbot is only allowed to answer using the documents you gave it. If the answer isn't in your documents, it will say *"I don't know"* instead of making something up. This makes it trustworthy enough to put in front of real customers.

The Two Sides of CreBot

1

The Dashboard (for the business owner)

A private control panel where you create bots, upload documents, customize how the chatbot looks and talks, and review every conversation it has had.

2

The Widget (for the website visitor)

A tiny, lightweight chat bubble that sits on the business's real website and lets any visitor talk to the trained chatbot in real time.

Company Vision

CreBot's stated mission is "**Absolute Clarity**" — the idea of turning messy, scattered company documents into one organized knowledge engine, so teams stop wasting time answering the same questions over and over, and customers get instant, correct answers.

Who Is It For? & The Problem It Solves

Any business that has written documentation and gets repetitive questions from customers or employees can use CreBot. Here are the main real-world situations it's built for:

1. SaaS Customer Support

Software companies get the same questions again and again about pricing, features, or how to fix a common problem. CreBot answers these instantly using the company's existing help docs — cutting down support tickets by roughly **40%**, working 24/7, and handing off to a human when needed.

2. Internal Knowledge & Operations

Instead of digging through a messy company wiki, employees can just *ask* the chatbot. It shortens onboarding time for new hires and becomes a single, reliable source of truth for internal information.

3. E-Commerce FAQs & Sales

Online stores use it to instantly answer questions about shipping, returns, and product specifications — helping turn browsers into buyers before they leave the page.

4. API & Developer Docs

Developer-facing companies let programmers ask questions directly against their technical documentation, including generating code snippets and finding the right API endpoint.

Other Great Fits

- Small businesses that can't afford a full-time support team.
- Solo entrepreneurs who need round-the-clock coverage without hiring anyone.
- Agencies who manage support chat for many different clients from one place.

Why this matters for a presentation: CreBot isn't a toy demo — it targets a real, expensive business problem (support cost and response time) and solves it with a self-serve, no-code product.

Key Features

1

Intelligent Chatbot Creation (RAG)

Every chatbot is fully grounded in the documents its owner uploads, using a technique called Retrieval-Augmented Generation (RAG) — see the Glossary for a full explanation.

2

Knowledge Base Management

Upload text, Markdown, PDF, or Word documents. CreBot automatically converts them into "vector embeddings" so the AI can search them intelligently.

3

Embeddable Chat Widget

A dependency-free JavaScript widget (`crebot-widget.js`) that can be pasted into any website with one script tag — no framework required.

4

Chat Logs & History

Every conversation is recorded. Owners can review exactly what visitors asked and how the bot answered, in real time.

5

API Access & Data Export

Generate secure API keys to connect CreBot into other tools, and export your full account data as JSON whenever you want.

6

Authentication & Security

Sign-up, sign-in, and session management are handled by Clerk. Every account's data is fully isolated — nobody can ever see another user's knowledge base.

7

Team Collaboration

Owners can invite teammates with either full Edit access or read-only View access to a specific bot.

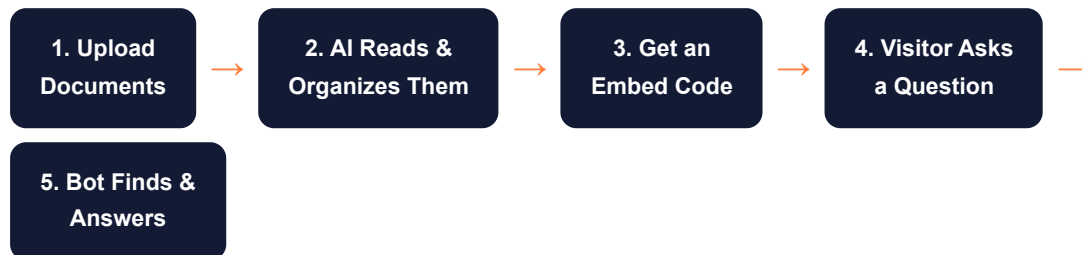
8

Bring Your Own Key (BYOK)

Power users can plug in their own free Groq API key to remove the shared rate limit and get much faster responses.

How CreBot Works — The Big Picture

Forget the code for a second. Here is the entire product in five plain-English steps:



Step 1 — Upload

The business owner uploads their documentation (a PDF, a Word doc, plain text, or Markdown) into the CreBot dashboard.

Step 2 — The AI Organizes It (Training)

Behind the scenes, CreBot breaks the document into small chunks, then converts each chunk into a list of numbers called a **vector embedding** — a mathematical fingerprint of what that chunk *means*. These are stored in a special database built for fast similarity search.

Step 3 — Get an Embed Code

Once training finishes, the owner copies a small HTML snippet from the "Embed" tab and pastes it into their own website's code.

Step 4 — A Visitor Asks a Question

Anyone visiting that website can click the chat bubble and type a question, just like texting a person.

Step 5 — The Bot Finds the Answer and Replies

CreBot converts the visitor's question into the same kind of "fingerprint," searches the stored document chunks for the closest matches, and hands those matching chunks to the Groq AI model, which is instructed to answer *using only that information*. If nothing matches well enough, it honestly says it doesn't know.

Why this design is smart: The AI never has to "remember" every document — it just needs to find the right small piece of text at the right moment and read it out loud in a natural way. This keeps answers accurate and lets the knowledge base scale to huge amounts of documentation.

Technology Stack, Explained Simply

Frontend (What the User Sees)

TECHNOLOGY	PLAIN-ENGLISH ROLE
React 18 + Vite	The framework that builds the actual dashboard screens, and the tool that runs/bundles the code very fast during development.
TypeScript	A stricter version of JavaScript that catches mistakes before the app even runs.
Tailwind CSS	A styling toolkit used to quickly design and theme every screen (colors, spacing, layout).
Framer Motion	Adds smooth animations, e.g. the animated mock chat on the homepage.
Clerk React	Ready-made, secure login/sign-up screens and session handling.
React Router	Controls which page shows up for which web address (URL).
Lucide React	The icon set used throughout the interface.

Backend (The "Brain" Running on the Server)

TECHNOLOGY	PLAIN-ENGLISH ROLE
FastAPI (Python)	A high-performance framework that powers all the behind-the-scenes logic and API endpoints.
Supabase (PostgreSQL)	The database that stores users, bots, documents, chat logs, and API keys.
pgvector	A database extension that lets Supabase store and instantly search AI "fingerprints" (vectors).
Groq API (Llama 3 models)	The actual AI model that reads the retrieved document chunks and writes the final answer, at very high speed.
SlowAPI	Enforces rate limiting so no single user can overload the shared servers.

Widget (The Public Chat Bubble)

Vanilla JavaScript — written with zero external dependencies so it never conflicts with a customer's existing website code, and stays extremely lightweight and fast to load.

SECTION 06

Authentication Flow (Clerk + JWT)

How CreBot knows who you are and keeps your data private.

CreBot uses a third-party service called **Clerk** to handle everything related to logging in — so the CreBot team never has to store your raw password themselves.

Step-by-Step

1. You sign in or sign up through Clerk's ready-made screens inside the React app.
2. Clerk gives your browser a signed digital pass called a **JWT (JSON Web Token)**.
3. Every time the dashboard asks the backend for data, it attaches this token in the request's `Authorization` header, as a "Bearer" token.
4. On the backend, a piece of code called `workspace_middleware` intercepts every incoming request and checks this token is genuine, using Clerk's public verification keys (JWKS).
5. Once verified, the backend extracts your unique Clerk user ID.
6. It then checks: does this user already have a "workspace" (their private account/tenant) in the database? If not, one is automatically created for them, named "My Workspace."
7. Your user ID and workspace ID are attached to the request so every other part of the backend automatically knows exactly whose data it's working with.

Key Files Involved

FILE	PURPOSE
<code>frontend/src/main.tsx</code>	Wraps the whole React app in Clerk's <code>ClerkProvider</code> .
<code>frontend/src/lib/api.ts</code>	Automatically attaches the Clerk JWT to every outgoing API request.
<code>backend/middlewares/auth.py</code>	Protects API routes and figures out which workspace the request belongs to.
<code>backend/utils/clerk_auth.py</code>	Decodes and validates the Clerk JWT using PyJWT.

Why it matters: This design means no page of code, anywhere in the app, has to re-invent login security — and every single database call is automatically scoped to the correct, isolated user account.

Frontend Architecture — Every Page & Component

The dashboard is built with React, TypeScript, Vite and Tailwind CSS. Below is exactly what each part of the frontend does.

Pages ([src/pages/](#))

FILE	WHAT IT DOES
LandingPage.tsx	The public marketing homepage — hero section, features, pricing.
DashboardOverview.tsx	The main dashboard home: bot counts, files uploaded, recent activity.
ChatbotsPage.tsx	Lists every chatbot the user has created.
ChatbotDetail.tsx / BotDetail.tsx	Configure a specific bot's name, avatar, prompt and appearance.
KnowledgeBasePage.tsx	Upload and manage the source documents that train a bot.
EmbedPage.tsx	Generates the script tag and settings used to embed the widget elsewhere.
ChatLogsPage.tsx	Shows the full history of conversations end-users had with a bot.
SettingsPage.tsx	Manage API keys, export account data, and dangerous actions like deletion.
BillingPage.tsx	Subscription / billing management screen (Stripe-based).
SignInPage.tsx / SignUpPage.tsx	Authentication wrappers around Clerk's UI components.
UserProfilePage.tsx	Personal account configuration.
AdminPage.tsx	System administrator dashboard.
JoinBotPage.tsx	Lets a teammate join an existing bot using a share code.

Components ([src/components/](#))

FOLDER	CONTAINS
layout/	Structural pieces: <code>Navbar.tsx</code> , <code>Sidebar.tsx</code> , <code>DashboardLayout.tsx</code> , <code>AuthLayout.tsx</code> .
chatbots/	Bot-specific UI: <code>ChatbotCard.tsx</code> , <code>CreateChatbotModal.tsx</code> , <code>UsageLimitBanner.tsx</code> .
common/	Reusable pieces used everywhere: <code>Button.tsx</code> , <code>Card.tsx</code> , <code>Input.tsx</code> , <code>Modal.tsx</code> , <code>Select.tsx</code> .
dashboard/	Widgets like <code>StatCard.tsx</code> and <code>RecentActivity.tsx</code> .
knowledge/	<code>UploadDropzone.tsx</code> and <code>KnowledgeTable.tsx</code> for managing documents.
landing/	Public page sections: <code>HeroSection.tsx</code> , <code>FeatureGrid.tsx</code> , <code>FAQSection.tsx</code> , <code>SecuritySection.tsx</code> .

Supporting Files

FILE	PURPOSE
<code>src/App.tsx</code>	Main router — decides which page loads for which URL, and separates protected vs. public routes.
<code>src/lib/api.ts</code>	One central place that every part of the app uses to talk to the backend.
<code>src/lib/clerkTheme.ts</code>	Makes the Clerk login screens visually match the rest of the app.
<code>src/context/ThemeContext.tsx</code>	Controls switching between Light and Dark mode.
<code>src/index.css</code>	Global styling rules and Tailwind setup.

Frontend Data Flow

Every screen follows the same simple pattern:

1. A page component (e.g. `ChatbotsPage.tsx`) loads.
2. It calls a function from the central `src/lib/api.ts` client.
3. That client automatically attaches the user's Clerk login token and sends the request to the FastAPI backend.
4. The page stores the response using normal React state (hooks) and renders it on screen.

Routing is handled with `react-router-dom` inside `App.tsx`, using lazy loading so pages are only downloaded by the browser when they're actually needed — keeping the app fast. Protected pages are wrapped in a `<SignedIn>` check, and public pages (like the landing page or sign-in) are wrapped in `<SignedOut>`.

Simple analogy: Think of `api.ts` as the app's single "receptionist." No matter which page needs information from the server, they all go through the same front desk — which makes the whole app easier to secure and maintain.

Backend Architecture — Every Route & Service

The backend is written in Python using FastAPI. It's split into two clean layers: **Routes** (which handle the web requests) and **Services** (which contain the actual business logic).

Core Files

FILE	PURPOSE
<code>main.py</code>	The application's entry point — sets up middleware, rate limiting, connects all the routers, and serves the widget's static files.
<code>config.py</code>	Loads environment variables and settings such as Supabase keys, the Groq API key, and Clerk configuration.
<code>models/schemas.py</code>	Defines the exact shape of data the API expects and returns (using Pydantic), for validation and auto-generated docs.

Routers (`backend/routes/`)

FILE	HANDLES
<code>bots.py</code>	Create, read, update, delete (CRUD) operations for chatbots.
<code>knowledge.py</code>	Document upload, parsing, and embedding generation.
<code>chat.py</code>	The dashboard's own "test chat" endpoint, used while configuring a bot.
<code>widget.py</code>	The public, rate-limited endpoint the live embedded widget actually talks to — the heart of the RAG chat logic.
<code>embed.py</code>	Returns the widget script/settings needed to embed a bot on an external site.
<code>settings.py</code>	API key generation, revocation, and full data export.
<code>dashboard.py</code>	Aggregates statistics (bot counts, storage used, etc.) for the frontend dashboard.
<code>chatlogs.py</code>	Retrieves historical conversation data for the Chat Logs page.

Services ([backend/services/](#))

FILE	PURPOSE
<code>groq_service.py</code>	Talks to the Groq API — rewrites vague follow-up questions into standalone questions, then generates the final answer.
<code>knowledge_service.py</code>	Manages stored file records and kicks off the chunking/embedding pipeline.
<code>chunking.py</code>	Splits long raw text into smaller, slightly overlapping chunks that are easier to search and embed.
<code>embedding.py</code>	Converts each text chunk into a vector embedding using a HuggingFace model.
<code>retrieval.py</code>	Performs the actual semantic (meaning-based) search in Supabase to find the most relevant chunks for a question.
<code>rate_limiter.py</code>	Enforces how many requests a user/bot can make per minute.
<code>settings_service.py</code>	Logic behind managing API keys and workspace-level configuration.
<code>dashboard_service.py</code> / <code>chatlog_service.py</code>	Database logic that powers the dashboard statistics and chat log pages.

Why Split "Routes" from "Services"?

This is a common, senior-level backend pattern, and it's worth understanding for a presentation:

R

Routes = "The Front Door"

These files only handle the web-facing parts: reading the incoming request, checking permissions, and returning a response. They stay short and easy to read.

S

Services = "The Engine Room"

These files contain the real logic — database queries, AI calls, text processing — completely separate from any web/HTTP details.

Why it matters: This separation means the same business logic (e.g. "search the knowledge base") can be reused by multiple routes without duplicating code, and it's much easier to test and maintain as the app grows.

External Services CreBot Depends On

SERVICE	ROLE
Clerk	User registration, login, session management, and securing API endpoints via JWT.
Supabase	The PostgreSQL database, plus <code>pgvector</code> for storing and querying AI embeddings.
Groq	Ultra-fast LLM inference — reformulating questions and generating the final grounded answer.

SECTION 09

Database Schema (Supabase + pgvector)

CreBot's entire "memory" lives in a PostgreSQL database, supercharged with the pgvector extension for AI-style semantic search.

TABLE	WHAT IT STORES
<code>workspaces</code>	One row per user account/tenant — includes the Clerk user ID, plan, and limits like chatbot/storage caps.
<code>bots</code>	Each chatbot's configuration: name, unique widget key, appearance (theme, welcome message), allowed domains, and a "strict knowledge" toggle.
<code>knowledge_files</code>	Metadata about every uploaded file: name, processing status, and how many chunks it produced.
<code>knowledge_chunks</code> / <code>documents</code>	The actual small text segments and their vector embeddings (384-dimension) used for search, per bot.
<code>chat_logs</code> / <code>queries_log</code>	Every question asked, the answer given, and which document sources were used to generate it.
<code>bot_members</code>	Tracks which teammates have Edit or View access to a shared bot.
<code>api_keys</code>	Programmatic access keys generated by users.

Row Level Security (RLS)

Every one of these tables is protected using PostgreSQL's built-in **Row Level Security**. In simple terms: even if a backend query has a mistake in it, the database itself will still refuse to return another user's rows. This is a second, independent safety net beyond the application code.

Semantic Search Function

A custom Postgres function called `match_documents` calculates **cosine similarity** between a question's vector and every stored chunk's vector, instantly returning the closest, most relevant matches.

The Embeddable Widget

The widget is the small chat bubble that actually appears on a customer's real website — the public face of every bot CreBot builds.

Why It Exists

Business owners shouldn't need to know React, or build any UI themselves, just to put a chatbot on their site. A single `<script>` tag is the simplest, most universal way to add functionality to any website — regardless of what that website was built with.

What `crebot-widget.js` Actually Does

- Injects its own CSS directly into the host page's `<head>`, so it doesn't rely on any external stylesheet.
- Builds all of its own HTML elements in JavaScript — the floating bubble and the chat panel.
- Manages every visual state: opening, closing, the "typing..." animation, and copy buttons.
- Sends the visitor's questions to the backend at an endpoint shaped like `API_URL/api/widget/{id}/chat`, identified using a `bot_id` or `widget_key` passed in via the script tag's `data-` attributes.
- Keeps a short-term memory of the conversation in the browser, so follow-up questions make sense in context.

Design Choices That Matter



Zero Dependencies

Written in plain ("vanilla") JavaScript so it never clashes with a host site's existing React, Vue, or jQuery code.



Style Isolation

Its own CSS/DOM is isolated from the host site's styling, so it always looks correct no matter where it's placed.



Live Configuration

Fetches the bot's name, color, and welcome message from the backend dynamically —



Cross-Device

Works the same way across desktop, tablet, and mobile.

dashboard changes appear instantly on the live widget without needing to re-embed anything.

SECTION 11

Complete Folder & File Structure

Everything in the project, in one place, so you know exactly where every piece of logic lives.

```
CreBot/
├─ schema.sql                # Core Supabase PostgreSQL schema
├─ backend/
│   ├── main.py              # FastAPI app entry, middleware, router mounting
│   ├── config.py            # Environment variable / settings loader
│   ├── models/
│   │   └─ schemas.py        # Pydantic request/response validation models
│   ├── middlewares/
│   │   └─ auth.py           # Clerk auth + workspace-context middleware
│   ├── utils/
│   │   └─ clerk_auth.py     # JWT decode/verify helpers
│   ├── routes/
│   │   ├── bots.py          # Chatbot CRUD
│   │   ├── knowledge.py     # Upload / parse / embed documents
│   │   ├── chat.py          # Dashboard test-chat endpoint
│   │   ├── widget.py        # Public widget chat endpoint (RAG core)
│   │   ├── embed.py         # Widget script/settings endpoint
│   │   ├── settings.py      # API keys + data export
│   │   ├── dashboard.py     # Dashboard statistics
│   │   └─ chatlogs.py       # Conversation history retrieval
│   ├── services/
│   │   ├── groq_service.py  # Talks to the Groq LLM
│   │   ├── knowledge_service.py # File records + pipeline trigger
│   │   ├── chunking.py      # Splits text into chunks
│   │   ├── embedding.py     # Text → vector embeddings
│   │   ├── retrieval.py     # Vector similarity search
│   │   ├── rate_limiter.py  # Usage / plan limits
│   │   ├── settings_service.py # API key + workspace logic
│   │   ├── dashboard_service.py # Dashboard stat aggregation
│   │   └─ chatlog_service.py # Chat log queries
│   └─ requirements.txt      # Python dependencies
├─ frontend/
│   ├── src/
│   │   ├── main.tsx         # App entry — ClerkProvider setup
│   │   ├── App.tsx          # Router config, protected/public routes
│   │   ├── index.css        # Global styles + Tailwind directives
│   │   └─ lib/
│   │       └─ api.ts        # Central API client (attaches JWT)
```

```
| | | └─ clerkTheme.ts      # Clerk UI theme matching
| | └─ context/
| |   └─ ThemeContext.tsx  # Light/Dark mode
| | └─ pages/              # 14 top-level screens (see Section 07)
| |   └─ components/
| |     └─ layout/         # Navbar, Sidebar, DashboardLayout
| |     └─ chatbots/       # Bot cards, create modal, usage banner
| |     └─ common/         # Button, Card, Input, Modal, Select
| |     └─ dashboard/      # StatCard, RecentActivity
| |     └─ knowledge/      # UploadDropzone, KnowledgeTable
| |     └─ landing/        # Hero, FeatureGrid, FAQ, Security sections
| └─ package.json          # Node dependencies
| └─ tailwind.config.js    # Styling rules / design tokens
└─ widget/
    └─ crebot-widget.js     # The deployable, dependency-free JS widget
```

Step-by-Step: From Upload to Answer

A plain-English walkthrough of the project's real sequence diagram.

Phase 1 — Training a Bot

1. The business owner uploads a knowledge base (FAQ text or files) from the React dashboard.
2. The dashboard sends this to the backend's training endpoint.
3. The backend chunks the text into paragraphs.
4. Each chunk is turned into a 384-number vector by the local embedding model.
5. The text chunks and their vectors are saved into Supabase.
6. The dashboard shows "Training Complete."

Phase 2 — A Visitor Chats With the Bot

1. A website visitor types a question into the embedded widget (e.g. *"What is your refund policy?"*).
2. The widget sends the question, plus recent chat history, to the backend's public widget-chat endpoint.
3. The backend checks the rate limit for that bot.
4. The Groq LLM rewrites the question into a clear, standalone version (using the conversation history for context).
5. That standalone question is converted into a vector.
6. Supabase performs a similarity search and returns the top matching chunks from the knowledge base.
7. Those chunks are sent to the Groq LLM again, with strict instructions to answer *using only this information*.
8. The final answer is logged for analytics, then sent back to the widget and displayed to the visitor.

The key insight: The AI is called *twice* per question — once to clean up the question itself, and once to generate the grounded answer. This two-step process is a big part of why CreBot's answers stay accurate even across a multi-turn conversation.

Security & Privacy

Data Isolation

- **Row Level Security (RLS)** at the database level — even a broken or malicious backend query cannot leak another user's data.
- **Client-isolated vector index with strict RBAC** — a widget key that belongs to one bot can never reach another bot's knowledge base or another tenant's data.

Authentication & Personal Data (PII)

Clerk securely manages all personally identifiable information — emails, passwords, and sessions. CreBot's own database only ever stores a *reference* to the authenticated user ID, never the raw credentials.

Prompt Injection Prevention

System prompts are rigidly structured so the AI model is explicitly instructed not to answer outside the provided document context, and to ignore any attempt by a user to override its prior instructions (a technique called **prompt injection** — see Glossary). CreBot is honest that no LLM-based system can guarantee 100% prevention of this — it's a strong mitigation, not an absolute guarantee.

BYOK Key Security

If you bring your own Groq API key, it is **AES-256 encrypted directly on your device** before it's ever sent to CreBot's servers — a "zero-knowledge" design, meaning CreBot itself never has access to your raw key.

Presentation tip: Security isn't just one feature — it's layered across four independent levels: the authentication provider (Clerk), the middleware, the database (RLS), and the AI prompt design itself.

SECTION 14

Access Control & Team Roles

A bot doesn't have to be used by just one person. CreBot lets an owner invite teammates with different levels of access to a specific bot:

ROLE	CAN DO
OWNER	Full control over the bot, including sharing it and managing who else has access.
EDIT	Full access to scan the knowledge base, write configuration, and edit bot parameters.
VIEW	Can view the bot, chat with it, and copy the embed code — but cannot change any configuration.

This makes CreBot practical for real teams and agencies: an admin can configure a bot in depth, while non-technical teammates (or clients) only get a safe, read-only way to demo or use it.

Rate Limits, BYOK & Pricing

Current Plan — Free for Everyone

CreBot is currently in a free-access phase for all sign-ups, with no credit card required. All core features are unlocked: bot creation, knowledge base training, widget deployment, chat logs, analytics, and access sharing.

LIMIT	VALUE
Chatbots per account	Up to 5
Default shared API rate limit	10 requests / minute per user (fair-usage policy)
If limit is hit	Wait one minute, or use BYOK to remove the cap entirely

Bring Your Own Key (BYOK)

Groq API keys are free to generate from Groq's own console. Once added in Settings, all of that bot's traffic is routed directly through the user's own private quota, skipping CreBot's shared queue entirely — unlocking much faster generation speeds.

Planned Billing (Future)

CreBot will introduce paid subscription tiers in a future release. The expected direction (not yet finalized) is a free tier with the limits above, plus paid tiers that unlock more than 5 bots, higher message volumes, priority support, and advanced analytics. Existing free users will be notified in advance of any billing changes.

Roadmap — What's Coming Next



Paid Billing Tiers

Formal subscription plans beyond the current free-access phase.



Website / URL Scraping

Auto-ingest a knowledge base directly from a live website, instead of only uploaded files.



Multi-Language Support

Ask and receive answers in multiple languages.



Advanced Analytics

Sentiment analysis and automatic surfacing of "unanswered" questions to guide what to add next.



Human Handoff

Detects frustration or complexity in a conversation and transfers the chat to a live agent (e.g. via Slack or Discord).

SECTION 17

Quick Start Guide

How a developer would run the whole project locally, from scratch.

1. Database Setup (Supabase)

1. Create a free project at supabase.com.
2. Go to **Database** → **Extensions** and enable `vector` (pgvector).
3. Open the SQL Editor, paste the schema file's contents, and run it to set up tables, Row Level Security, and search functions.

2. Backend Setup

```
cd backend
python -m venv venv
venv\Scripts\activate      (Windows)
pip install -r requirements.txt
copy .env.example .env      (fill in Supabase, Groq, Clerk keys)
uvicorn main:app --reload --port 8000
```

3. Frontend Setup

```
cd frontend
npm install
npm run dev
```

4. Test the Widget

Paste this snippet into the `<body>` of any HTML file to test a trained chatbot:

```
<script
  src="http://localhost:3000/widget/crebot-widget.js"
  data-widget-key="YOUR_BOT_WIDGET_KEY"
  data-api-url="http://localhost:8000"
  async>
</script>
```

Glossary — Every Technical Word, Explained

Written for someone who has never studied software engineering. Read this section any time a word in this document is unfamiliar.

API (Application Programming Interface)

A defined way for two pieces of software to talk to each other — like a waiter taking your order to the kitchen and bringing food back.

RAG (Retrieval-Augmented Generation)

A technique where the AI first "retrieves" (looks up) relevant information, then "generates" an answer using only that information, instead of relying purely on what it memorized during training.

LLM (Large Language Model)

The type of AI model (like Llama 3) that has been trained on huge amounts of text and can read, understand, and write natural-sounding language.

Hallucination

When an AI confidently states something false or made-up. CreBot's whole design is aimed at preventing this.

Vector / Embedding

A list of numbers that represents the "meaning" of a piece of text. Two chunks of text about similar topics will have vectors that are mathematically close together, which is how the AI finds relevant information quickly.

Vector Database (pgvector)

A database built (or extended) to store and instantly search these number-lists by similarity, rather than by exact matching text.

Chunking

Breaking a long document into smaller pieces so the AI can search and reference specific, focused sections instead of an entire book at once.

Cosine Similarity

A mathematical way of measuring how "close" two vectors are in meaning — the method used to find the best-matching document chunks for a question.

JWT (JSON Web Token)

A digitally signed "ID card" issued after login, proving who a user is on every future request, without needing to log in again each time.

Middleware

Code that runs automatically in between a request arriving and the actual page/feature responding to it — commonly used for things like security checks.

RBAC (Role-Based Access Control)

A security model where what you're allowed to do depends on your assigned role (e.g. Owner, Edit, View), rather than being the same for everyone.

RLS (Row Level Security)

A database-level rule that automatically hides other users' data rows, even if the application code above it makes a mistake.

Prompt Injection

An attack where someone tries to trick an AI into ignoring its original instructions by hiding new "commands" inside their message.

SaaS (Software as a Service)

Software that you use directly through a website, paying (or signing up) for access, instead of installing and running it yourself.

CRUD

Short for Create, Read, Update, Delete — the four basic operations almost every app needs to perform on its data.

Widget

A small, self-contained piece of interface — here, the floating chat bubble that gets embedded on someone else's website.

Rate Limiting

Deliberately capping how many requests a user can make in a given time period, to keep the shared service fast and fair for everyone.

BYOK (Bring Your Own Key)

Letting a user supply their own API key from a provider (here, Groq) so their usage runs on their own private quota instead of the shared one.

AES-256 Encryption

A very strong, widely trusted method of scrambling data so it can only be read by someone with the correct decryption key.

Framework

A pre-built structure and set of tools (like React or FastAPI) that developers build on top of, instead of writing everything completely from scratch.

Tenant / Multi-Tenancy

A design where one shared system serves many independent customers ("tenants") while keeping each one's data fully separate and private.

Endpoint

One specific web address that a piece of software (like the widget) can send a request to in order to get something done, e.g. `/api/widget/{id}/chat`.

Session Management

How an application keeps track of the fact that you're still logged in as you move between pages.

Environment Variables

Configuration values (like secret API keys) kept outside of the code itself, so sensitive information is never hard-coded or accidentally shared.

Static Files

Files (like the widget's JavaScript file) that are served exactly as-is, without needing to be processed by the backend each time.

Backend / Frontend

The Frontend is everything the user directly sees and clicks on in the browser. The Backend is the server-side code that stores data, applies business rules, and talks to the AI — invisible to the user.

Repository / Codebase

The complete collection of source code files that make up the project.

Deployment

The process of taking finished code and making it live and accessible on the real internet.

Company & Contact

DETAIL	INFO
HQ Location	Gandhinagar, Gujarat, India
Core Team / System Architects	Priyank, Miral, Sujal
Contact	Available via the website contact form — for enterprise plans, custom integrations, or proof-of-concept discussions
Typical Response Time	Within 4 hours
Legal	Privacy Policy, Terms of Service, and Cookie Policy published and linked in the site footer

Closing Summary

CreBot is a complete, production-shaped SaaS product: a secure multi-tenant dashboard, a Zero-Speculation RAG pipeline built on vector search, a lightweight universal widget, and a layered security model — all wrapped in a free, no-code onboarding experience for any business that wants an AI support agent trained on their own documentation.

End of Document. Thank you for reading — this documentation was written to be presentable to technical reviewers, business stakeholders, and complete beginners alike.